

Title Page

BantayAralan: Real-Time Student Behavior and Classroom Orderliness System

Chloe Lane B. Jose
Raynier Ronn G. Santos
Justin Rieko S. Tolentino

Bachelor of Science in Computer Engineering
School of Engineering and Architecture
Holy Angel University

Master Document ID: DOC-CAP-FINAL-2026-008
Date of Issue: September 26, 2026

Document Revision History

Version	Date	Author(s)	Description of Changes
0.1	September 15, 2026	Team 8	Initial project documentation
0.2	September 15, 2026	Team 8	Integration of requirements and design specifications
0.3	September 16, 2026	Team 8	Hardware and software architecture revisions
0.4	September 17, 2026	Team 8	Integration of AI, privacy, and database specifications
0.5	September 24, 2026	Team 8	Finalized camera model (TP-Link VIGI C320I, PoE), detection model (YOLOv8), and local hosting plan; added Sections 2.1–2.6 and 3.0
1.0	September 26, 2026	Team 8	Final consolidated capstone architecture document

Approval Signatures Block

Name	Role	Signature	Date
Jose, Chloe Lane B.	Student Author	_____	_____
Santos, Raynier Ronn G.	Student Author	_____	_____
Tolentino, Justin Rieko S.	Student Author	_____	_____
[Name]	Technical Reviewer / Project Manager	_____	_____
[Name]	Academic Adviser	_____	_____

Table of Contents

Document Revision History..... ii

Approval Signatures Block ii

Table of Contents..... iii

Glossary v

2.1 High-Level Architecture 1

 System Overview — Block Diagram..... 1

 Major Software Modules 1

 Data Flow 2

2.2 Detailed Software Architecture 3

 UML Class Diagram (Module View)..... 3

 UML Sequence Diagrams 3

 UML Use Case Diagram..... 4

 Database Design 4

 Entity-Relationship Diagram 4

 Schema, Normalization, and SQL vs. NoSQL Justification 4

 System APIs and Protocols 5

2.3 Algorithms and Data Structures 6

 Core Algorithms 6

 Data Structures..... 6

 Design Patterns & Maintainability 6

2.4 AI System Design 8

 Training Data & Data Engineering 8

 Model Architecture Selection 8

 Performance Metrics & Verification..... 9

 Deployment Plan & Sustainability 9

 Regulatory, Legal & Ethical Compliance 9

2.5 Hardware-Software Interface Specification 11

 Hardware Architecture..... 11

 Data Ingestion & Telemetry..... 11

 Edge Compute & Firmware Constraints 12

 Actuation & Control Logic 12

2.6 Source Code & Version Control Strategy 13

 Repository Architecture 13

 Branching Workflow 13

 Commit & Merge Conventions..... 13

CI/CD Pipeline.....	13
3.0 Final Master Document Compilation	14
Completeness Verification	14
Architectural Integration	14
Historical Aggregation.....	14
Formatting Integration	14
Outstanding Items Before Final Submission	14

Glossary

Term	Definition
AI	Artificial Intelligence — computational systems performing tasks that normally require human intelligence.
ByteTrack	Multi-object tracking algorithm used to associate detected objects/persons across consecutive video frames.
Camera	TP-Link VIGI C320I — a PoE dome network camera.
Clutter / Out-of-Place Object	An item or area that is out of its expected place, determined by comparison against a reference layout of the room, not by classifying the type of object (e.g. “paper,” “bottle”).
Confidence Score	Numerical value representing a detection model's confidence in a given detection.
Database	SQLite — structured local storage for event, head-count, and detection-state records.
Event	A recorded occurrence of disruptive behavior, an out-of-place object, or misaligned furniture.
Event Log	The historical list of recorded events (‘events’ table / Events & Logs screen).
FPS	Frames Per Second — the number of video frames processed per second.
HITL	Human-in-the-Loop — the system supports, and does not replace, teacher review and decision-making.
Object Storage	Local file-based storage for evidence files (snapshots and video clips), hosted on-premises rather than a cloud storage service.
OpenCV	Open-source computer vision library used for video processing and image manipulation.
PoE	Power over Ethernet — supplies both power and network connectivity over a single Ethernet cable; used by the camera.
SQLite	Lightweight, file-based relational database management system used for all local data storage.
YOLOv8	Object-detection model family used for BantayAralan's real-time visual detection.
admin-ui	The BantayAralan admin interface — a Flask (Python) backend serving a plain HTML/CSS/JS frontend, run either as a browser tab or a native window.

2.1 High-Level Architecture

BantayAralan consists of two layers: a computer-vision detection pipeline (camera capture, detection, tracking, and behavior/alignment logic) and an admin web application (event review, head count, and reporting). The admin application is implemented and running against seeded data; the detection pipeline is designed but not yet built. The diagram below shows both layers and how they connect.

System Overview — Block Diagram

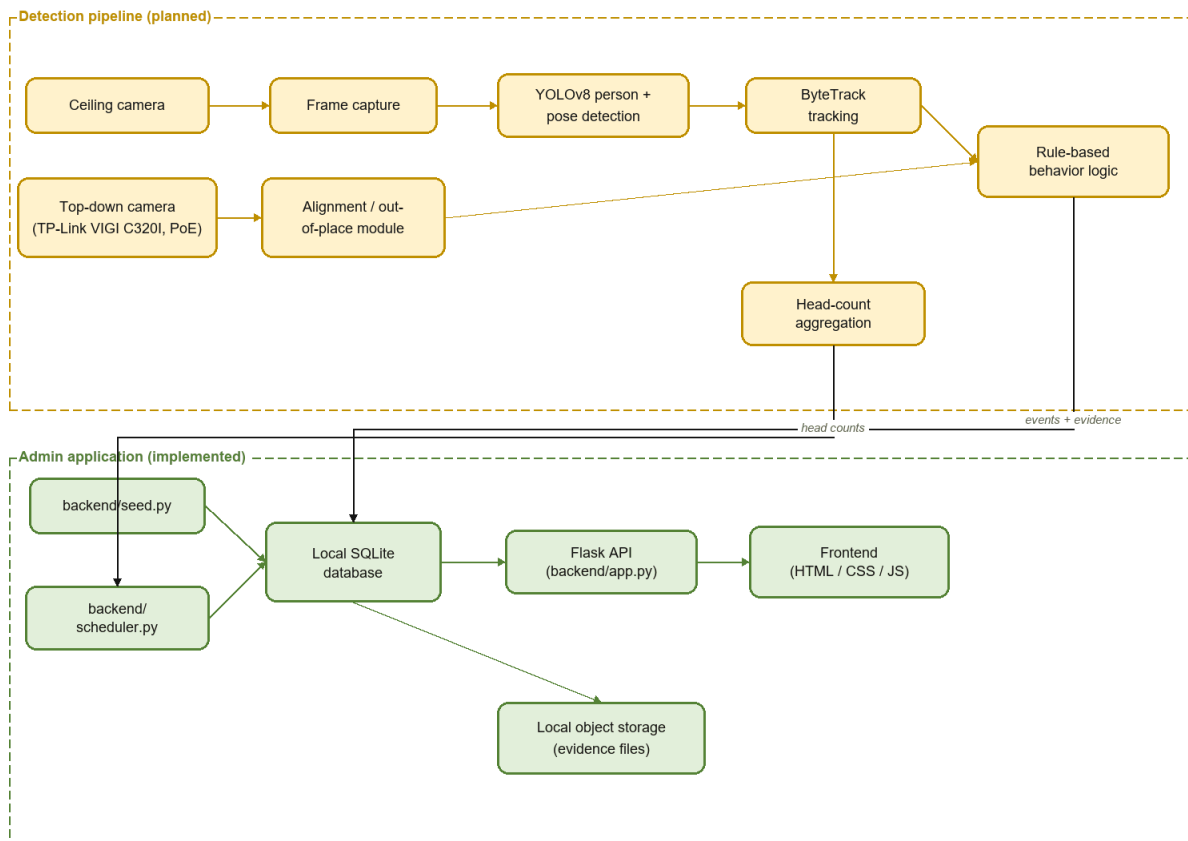


Figure 2.1 — System block diagram: the detection pipeline (planned, orange) and the admin application (implemented, green), with the two points where the pipeline writes into the admin application's local storage.

Local database (SQLite) and local object storage (evidence files) are hosted on the same on-premises machine as the application; no cloud database or cloud storage bucket is used.

Major Software Modules

- **admin-ui/backend/** — Flask application: routing, JSON serialization, statistics/insight computation, database access, background scheduler.
- **admin-ui/frontend/** — static HTML/CSS/JS single-page application; no build pipeline, no framework.

- **Detection pipeline** — camera capture, YOLOv8 detection and pose estimation, ByteTrack tracking, rule-based behavior and alignment logic. Planned; not yet built.
- **Database** — SQLite, hosted locally, three tables (see Section 2.2).
- **Object Storage** — evidence files (snapshots and video clips) are stored on local disk on the same host as the application, not in a cloud storage bucket. This removes recurring cloud database and storage costs and keeps classroom footage on-premises.
- **User Interface** — one web application, run either as a browser tab or as a native window, sharing one Flask backend and one frontend.
- **Hardware Interface** — a TP-Link VIGI C320I PoE dome camera for camera input (see Section 2.5).

Data Flow

Admin application: `seed.py` populates the database at startup → `scheduler.py`'s background thread records head counts at the configured times → the frontend requests data from the Flask API → the API queries the local database and returns JSON → the frontend renders it.

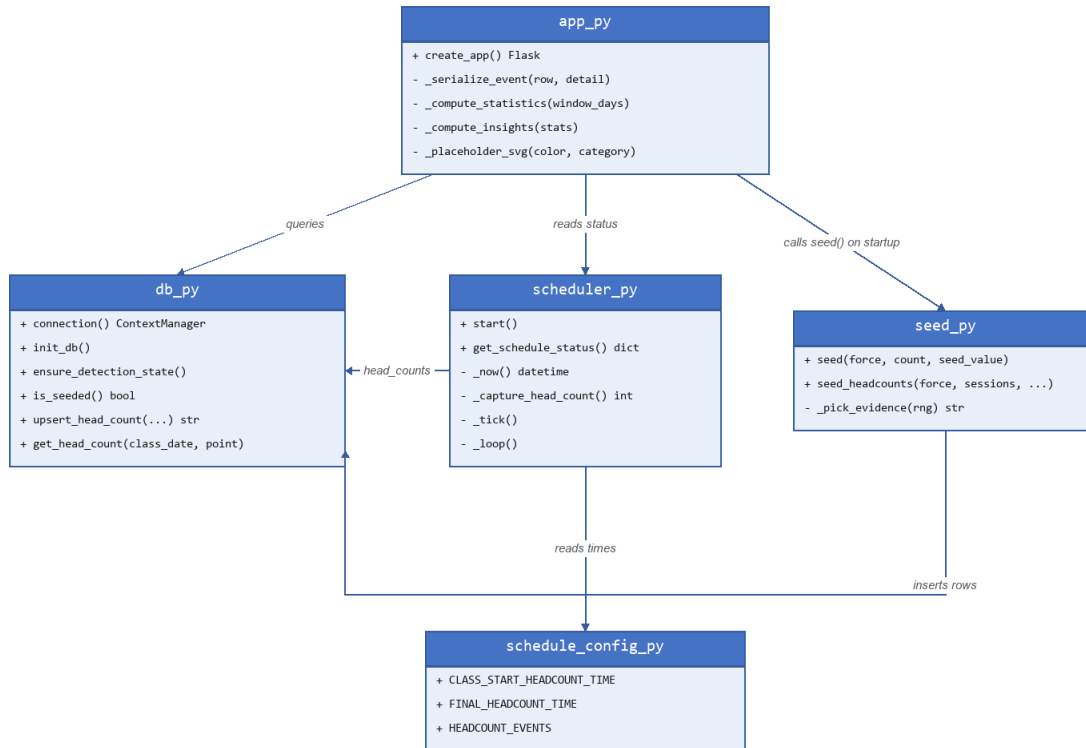
Detection pipeline (planned): camera frames → detection/pose (YOLOv8) → tracking (ByteTrack) → rule-based behavior/alignment decision → event and evidence written to the local database and local object storage → the admin application displays it.

Deployment model: the system runs on a single on-premises machine. The database and evidence storage are both hosted locally on that machine rather than in the cloud, avoiding cloud database and object-storage costs. There is no cloud component in the architecture.

2.2 Detailed Software Architecture

The backend (admin-ui/backend/) is organized as modules of functions rather than classes. The diagrams below present it as a module/responsibility view.

UML Class Diagram (Module View)



UML Sequence Diagrams

Flow 1 — Dashboard page load:

1. Browser (frontend/js) sends GET /api/summary to Flask.
2. Flask (app.py) runs SELECT COUNT(*), category breakdown, and recent-rows queries against the local database (db.py).
3. The database returns rows to Flask.
4. Flask returns a JSON body: {total_events, events_today, by_category, recent}.
5. The browser renders it via renderStats() / renderRecent() (components.js).

Flow 2 — Scheduler tick:

6. Every 15 seconds, scheduler._loop() calls _tick().
7. _tick() asks db.get_head_count(class_date, point) whether today's row already exists.
8. If it exists → status = “complete”. If the event's scheduled time hasn't arrived → status = “waiting”.

9. If the scheduled time has arrived while the app was running — status becomes “performing”, the head count is captured, and the result is written via `db.upsert_head_count(..., source="scheduled")`.
10. If the scheduled time passed before the app started, the event is marked “missed” instead of firing retroactively.

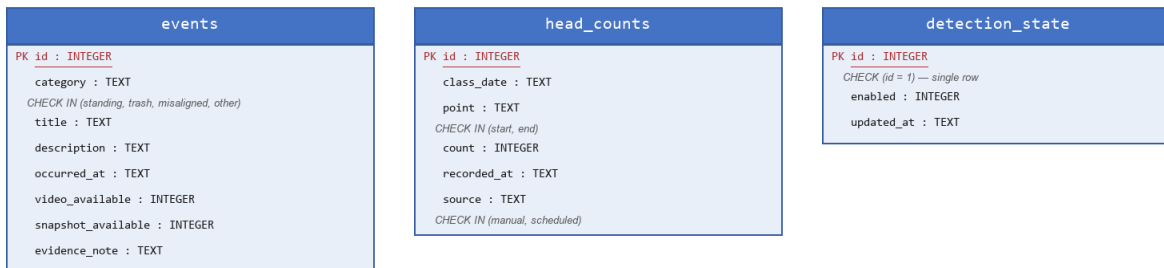
UML Use Case Diagram

Actor	Use Cases
Admin/Teacher	View dashboard summary; browse/filter/search/sort events; view an event's video/screenshot evidence; view today's head-count schedule and history; toggle detection enabled/disabled; view statistics, classroom insights, and suggestions; toggle light/dark theme
Scheduler (system actor)	Automatically trigger and record class-start and final head counts at configured times, with no human action

There is no login, user management, or system configuration use case — the application is single-role and admin-only by design.

Database Design

Entity-Relationship Diagram



No foreign-key relationships exist between the three tables. Each table models one independent fact, so no join is required across them.

Schema, Normalization, and SQL vs. NoSQL Justification

Technology: **SQLite 3**, accessed via Python's standard-library `sqlite3` module (no ORM), hosted locally rather than as a managed cloud database. Primary keys: `INTEGER PRIMARY KEY AUTOINCREMENT` on ``events`/`head_counts``; a singleton ``CHECK (id = 1)`` primary key on ``detection_state``. The schema is in third normal form: each table represents one independent fact (an event occurrence, a head-count reading, a single toggle state) with no repeating groups or transitive dependencies; enum-like columns (``category``, ``point``, ``source``) are constrained via ``CHECK`` rather than split into lookup tables, which would add joins without a normalization benefit at this scale.

SQL vs. NoSQL: a relational database is the appropriate choice, for three reasons. First, the data is inherently tabular and low-cardinality — three small, independently structured tables with simple filters (by category, date range, and text search), which is exactly what SQL's declarative query model and indexing are built for. Second, the application's query patterns (``WHERE category = ? AND occurred_at`

>= ?', pagination, ordering) map directly onto SQL, whereas most NoSQL document/key-value stores would require re-implementing equivalent filtering/sorting logic in application code. Third, SQLite needs no separate database server process, which matches the project's local, single-machine hosting plan and keeps the only runtime dependency at `pip install flask` — with no managed cloud database, there is no recurring hosting cost. A NoSQL store would only be worth reconsidering if the project moved toward high-volume, schema-flexible event streams (e.g. raw per-frame detection output) or multi-classroom horizontal scaling, neither of which is in scope.

System APIs and Protocols

The application exposes a **local, single-process Flask JSON API**. There is no GraphQL, gRPC, WebSocket, or message-queue interface.

Method	Route	Purpose
GET	/, /<path:path>	Serve the static frontend (index.html, JS/CSS assets)
GET	/api/summary	Dashboard totals, today's count, by-category counts, 6 most recent events
GET	/api/status	Camera/monitoring status, detection-enabled flag, last event time
GET / POST	/api/detection-state	Read / set the detection-enabled flag ({"enabled": bool})
GET	/api/headcounts	Head-count session history, grouped by class_date
GET	/api/headcount-schedule	Today's scheduled head-count events and their live status
GET	/api/statistics	7-day windowed event counts/trend
GET	/api/insights	Rule-derived observation strings
GET	/api/suggestions	Per-category templated suggestion strings
GET	/api/events	Filtered/sorted/paginated event list (category, range, q, sort, limit, offset)
GET	/api/events/<id>	Single event detail
GET	/api/events/<id>/snapshot.svg	Snapshot evidence image

Communication protocol: HTTP/1.1 with JSON bodies, over the local network (port 5057 for the browser-tab mode, port 5058 for the native-window mode). Authentication is not required, consistent with the single-admin, single-deployment scope. The camera (TP-Link VIGI C320I) is ingested over the local network via its standard PoE IP-camera protocol (RTSP/ONVIF).

2.3 Algorithms and Data Structures

The algorithms below are the ones implemented in `admin-ui/` today; detection, tracking, pose-estimation, and spatial-analysis algorithms belong to the planned detection pipeline (Section 2.4) and are not yet built.

Core Algorithms

Algorithm	Where	Time Complexity	Space Complexity
Weighted-random evidence-kind selection	<code>seed.py _pick_evidence()</code>	$O(k)$, $k=4$ (constant)	$O(1)$
Deterministic mock-data generation	<code>seed.py</code> <code>seed()/seed_headcounts()</code>	$O(n)$, n = seed size	$O(n)$
Dynamic filtered/paginated query	<code>app.py list_events()</code>	$O(n)$ per query	$O(1)$ beyond the LIMIT-bounded page
Session grouping	<code>app.py list_headcounts()</code>	$O(n)$ group + $O(n \log n)$ sort	$O(n)$
Idempotent scheduler tick	<code>scheduler.py _tick()</code>	$O(1)$ per tick (2 fixed events)	$O(1)$
Windowed statistics aggregation	<code>app.py</code> <code>_compute_statistics()</code>	$O(n)$ per query, constant query count	$O(1)$
Top-category / trend-threshold insight	<code>app.py</code> <code>_compute_insights()</code>	$O(k)$, $k=4$ (constant)	$O(1)$

Performance note: event queries run against unindexed columns (`'category'`, `'occurred_at'`). Adding indexes on these columns is recommended once the detection pipeline is producing continuous event volume; at the current data scale it has no measurable effect.

Data Structures

- `SUGGESTION_MAP`, `CATEGORY_LABELS`, `CATEGORY_ICON_COLOR` — dictionaries mapping category keys to display/behavior data.
- `_runtime_status` — an in-memory dictionary (2 entries) tracking each head-count event's live status, protected by a `threading.Lock`.
- `HEADCOUNT_EVENTS` — a list of dictionaries (key/point/time/label) driving both the scheduler and the schedule-status API.
- Database rows are accessed as `sqlite3.Row` objects (dictionary-like, column-name access).

Design Patterns & Maintainability

- **Application Factory** (`app.py`'s `create_app()`) — the app instance is built inside a function rather than at import time, keeping app construction/configuration testable.

- **Context manager / resource acquisition** (`db.py`'s `@contextmanager def connection()`) — guarantees every database connection is committed and closed via Python's `with` statement, preventing connection leaks as the codebase grows.
- **Polling** — the frontend's 20-second refresh of `/api/headcount-schedule` and the scheduler's own tick loop use timed polling rather than a push mechanism, keeping both sides simple and dependency-free at this scale.
- **Idempotent/stateless recomputation** — the scheduler derives status from the database on every cycle instead of a mutable “already fired” flag, so it resumes correctly after a restart or a new calendar day without extra state-management code.

2.4 AI System Design

This section defines the AI system to be built: the model architecture, training-data plan, target metrics, deployment approach, and compliance obligations for the detection pipeline.

Training Data & Data Engineering

The detection pipeline uses pretrained models for person detection, pose estimation, and tracking, and does not require a custom-trained model for its core functions. Out-of-place/mess detection is designed as a reference/zone comparison against the fixed top-down camera view, and head-count/behavior detection use pretrained person and pose detection — neither needs a custom dataset. Per the adviser's direction, the system determines whether an item or area is out of place or messy, and does not classify trash by type (“paper,” “bottle,” “plastic,” etc.).

If a custom dataset is developed for a future refinement

- **Purpose:** distinguish classroom material or debris that is out of place from a clean baseline, from the TP-Link VIGI C320I camera's field of view.
- **Source/provenance:** captured from the target classroom(s) with the actual camera and mounting, since camera angle, lighting, and desk layout are classroom-specific.
- **Volume:** several hundred to low-thousands of annotated images per class, following standard fine-tuning practice.
- **Annotation:** bounding boxes around out-of-place items with a binary or few-class label (“in place” / “out of place”), not a fine-grained trash taxonomy.
- **Train/validation/test split:** a 70/15/15 or 80/10/10 split, stratified by lighting condition across capture sessions.
- **Data cleaning:** de-duplication of near-identical frames and removal of blurred or motion-corrupted captures.
- **Anonymization / bias mitigation:** captured frames avoid identifying individual students beyond what a teacher needs to review an event; the top-down camera angle reduces facial visibility by geometry. Class imbalance is expected, since a tidy classroom is the majority state, which favors an anomaly/zone-deviation framing over a balanced-class classifier.

Model Architecture Selection

Model	Architecture	Notes
YOLOv8	Single-stage CNN object and pose detector (Ultralytics YOLOv8 family)	Used pretrained (COCO weights) for person detection; variant (n/s/m/l/x) to be chosen based on target hardware
ByteTrack	Tracking-by-detection: Kalman-filter motion prediction with a two-stage (high- and low-confidence) IoU association step	Associates low-confidence detections, which helps track partially occluded students and reduces head-count double-counting

Alternatives considered:

- **Detector:** YOLO11m and two-stage detectors such as Faster R-CNN were considered. YOLOv8 is selected for its tooling maturity and real-time performance on the project's target hardware.
- **Tracker:** SORT (simpler, motion-only, more prone to ID switches under occlusion) and DeepSORT (adds an appearance re-identification embedding that improves occlusion handling but computes a soft-biometric-like feature per person) were considered. Motion-based ByteTrack is preferred, both for occlusion handling and because it avoids the identity-tracking implications of an appearance embedding.

Performance Metrics & Verification

Metric	Status	Target
Detection precision/recall/F1/mAP	To be measured in pilot	Set from the pilot baseline against a held-out annotated test set
Tracking ID-switch rate / MOTA / IDF1	To be measured in pilot	Quantifies head-count double-counting risk
Inference latency / FPS	To be measured in pilot	Confirms real-time operation on deployment hardware
CPU/GPU/memory utilization	To be measured in pilot	Sizes resource requirements on deployment hardware
Teacher usability (5-point Likert survey)	Not yet collected	Pre- and post-implementation, per the project's evaluation methodology

Precision and recall matter specifically because a classroom deployment cannot tolerate a high false-positive rate (flooding a teacher with false events erodes trust in the human-in-the-loop design) or a high false-negative rate (missed events defeat the system's purpose); mAP summarizes detector quality across confidence thresholds for exactly this trade-off. **Verification procedure:** pilot recordings from the actual classroom with the TP-Link VIGI C320I camera; a held-out annotated test set; COCO-style mAP for detection, MOTA/IDF1 for tracking, and wall-clock FPS/latency profiling on the deployment hardware.

Deployment Plan & Sustainability

- **Hosting strategy:** local/on-premises. The processing PC, the local database, and local object storage for evidence files all run on the same site as the classroom, with no cloud service in the loop. This removes recurring cloud database and storage-bucket costs.
- **Runtime:** the Ultralytics YOLOv8 family runs on a Python + PyTorch stack.
- **Hardware requirements:** real-time inference from YOLOv8 with ByteTrack is GPU-assisted; CPU-only operation uses a smaller model variant (YOLOv8n/s) or a reduced frame rate if needed.
- **Sustainability / resource constraints:** options to manage compute cost include a smaller YOLOv8 variant, cropping each frame to the classroom's region of interest before inference, and reducing frame rate outside active class hours.
- **Model loading / inference pipeline:** capture → detect → track → decide → log/store evidence → display, matching the flow in Section 2.1.

Regulatory, Legal & Ethical Compliance

The following table summarizes the system's alignment with the National Privacy Commission's pillars under the Data Privacy Act of 2012 (RA 10173). Hosting the database and evidence storage locally reduces third-party data-transfer exposure, but does not by itself satisfy the Act's other obligations, which are tracked separately below.

NPC Pillar / Safeguard	Current State
Transparency	A privacy notice and parental/guardian consent process are required before deployment in any classroom; the consent procedure is to be finalized with the adviser.
Legitimate Purpose	The system assists teacher awareness of classroom disorder and behavior; a formal legal/DPO review is planned before deployment.
Proportionality	Enforced by design: no facial recognition or identity tracking is implemented anywhere in the system.
Human-in-the-Loop (HITL)	The system supports, and does not replace, teacher decision-making; all detected conditions are surfaced as events for teacher review, not automated actions.
Explainable AI (XAI)	Each event is paired with its supporting evidence (screenshot/video) so a teacher can review the basis for it directly.
Accountability	Administrative actions (e.g. the detection toggle) are timestamped; expanding this to a full audit trail is planned if multi-user access is introduced.
Fairness	Detection/tracking accuracy will be evaluated across lighting conditions, seating positions, and clothing variation during the pilot phase, before deployment.
Data Minimization	Only frames within a triggering event's window are retained; continuous raw recording is not stored.
Retention	An evidence-retention policy is to be finalized with the adviser before deployment.

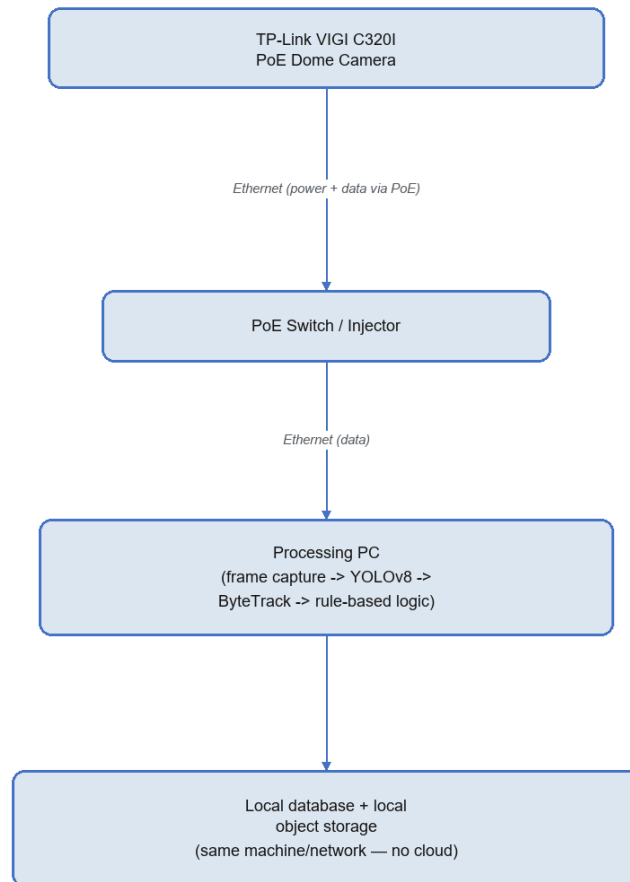
ISO/IEC 42001 (AI management system) lifecycle mapping:

Lifecycle Stage	Plan
Context & scope establishment	Defined in the research proposal and this document
AI risk assessment	To be completed before pilot deployment
System requirements	Documented in Sections 2.1–2.4
Design & development	In progress
Verification & validation	Pilot evaluation per Section 2.4's performance metrics
Deployment	Local/on-premises, per Section 2.4
Operation & monitoring	Ongoing once deployed
Post-deployment impact assessment	Scheduled after pilot deployment

2.5 Hardware-Software Interface Specification

BantayAralan is a camera-and-computer vision system, not an embedded/microcontroller system. Pin mapping, I2C/SPI buses, firmware quantization, and actuation therefore do not apply to this project and are addressed briefly below.

Hardware Architecture



There is no pin-mapping table, since the camera connects over a standard Ethernet/PoE network link rather than a wired peripheral bus, and no microcontroller or embedded board is used anywhere in the system.

Data Ingestion & Telemetry

- **Communication protocol:** the TP-Link VIGI C320I camera streams over RTSP/ONVIF, standard for this class of PoE IP camera.
- **I2C / SPI / UART:** *Not applicable.* These are embedded-device peripheral buses; the camera is a standalone network device.
- **Buffer management:** a rolling per-camera frame buffer retains recent frames so a captured clip can include time before the triggering event.

Edge Compute & Firmware Constraints

Not applicable. The system runs on a general-purpose processing PC, not an embedded device, so there is no firmware to constrain and no model quantization step for an embedded target. If the processing PC is resource-constrained, a smaller YOLOv8 variant (n/s) is used instead of an embedded-style quantization approach.

Actuation & Control Logic

Not applicable. BantayAralan detects and logs conditions for teacher review; it does not control any physical actuator. Its output is data: a recorded event, a stored head count, and a display in the admin application.

2.6 Source Code & Version Control Strategy

Repository Architecture

- **Hosting platform:** GitHub.
- **Repository link:** <https://github.com/riekoqq/BantayAralan>
- **Structure:** a single repository containing the admin application and project documentation.

Branching Workflow

- **Default branch:** master.
- **Strategy:** changes are committed directly to `master`. A feature-branch workflow with pull-request review is planned as the team grows past the current prototype stage.

Commit & Merge Conventions

- **Commit message format:** descriptive, multi-line messages explaining what changed and why.
- **Authors:** Team 8 (Jose, Santos, Tolentino), with AI pair-programming assistance disclosed in commit trailers where used.
- **Merge rules:** pull-request review will be adopted once development moves beyond the current single-contributor prototype phase.

CI/CD Pipeline

- **Automated testing:** manual verification is used today (click-through testing of the admin UI, and a time-override test procedure for the scheduler). Automated tests are planned as the codebase grows.
- **Linting:** planned alongside the automated test suite.
- **CI/CD:** a GitHub Actions workflow for linting and tests is planned for the next development phase.

3.0 Final Master Document Compilation

Completeness Verification

Requirement	Status
2.1 High-Level Architecture	Complete
2.2 Detailed Software Architecture	Complete
2.3 Algorithms & Data Structures	Complete
2.4 AI System Design	Complete
2.5 Hardware-Software Interface	Complete
2.6 Source Code & Version Control Strategy	Complete
Historical milestone aggregation	In progress — see below
Approval signatures	Pending
Adviser review	Scheduled

Architectural Integration

Sections 2.1 through 2.6 form one integrated architecture: the confirmed camera (TP-Link VIGI C320I), detection model (YOLOv8), and local hosting plan for the database and evidence storage are used consistently across all six subsections.

Historical Aggregation

The following milestone documents are attached in Folder 2 of the final submission:

- Team8_BantayAralan-Proposal.docx — the original thesis proposal.
- BantayAralan-Prototype-Paper.docx — the revised prototype paper.

A Requirements Specification, Feasibility Study, Wireframes/UI-UX Prototypes, and Ethical Clearance are being compiled as standalone documents for Folder 2.

Formatting Integration

This document uses one consistent font (Times New Roman body text, Consolas for code and diagram-source blocks), one heading hierarchy feeding the automated Table of Contents, and one table style throughout. The same formatting standard is applied to the historical milestone documents when they are compiled into the final submission package.

Outstanding Items Before Final Submission

1. Obtain the Technical Reviewer/Project Manager's and Academic Adviser's names and signatures.
2. Complete adviser review and record the outcome.
3. Finalize the evidence-retention policy with the adviser.
4. Finalize the parental/guardian consent process.

5. Compile the Requirements Specification, Feasibility Study, Wireframes, and Ethical Clearance documents for Folder 2.
6. Confirm the camera role assignment (ceiling vs. top-down) and finalize the ingest protocol configuration.
7. Assemble Folders 2–4 of the final submission (milestone files; repository access confirmation, README, and source backup; demo video and presentation deck).